

Adversarial Attacks and Defenses on Graph-aware Large Language Models (LLMs)

Anonymous submission

Abstract

The accelerating integration of Large Language Models (LLMs) with graph-structured data for tasks such as node classification—a domain long dominated by Graph Neural Networks (GNNs)—harnesses rich relational information to elevate task performance, yet the adversarial robustness of the resulting architectures remains conspicuously underexplored. We undertake the first systematic interrogation of the vulnerabilities inherent to graph-aware LLMs, repurposing established adversarial attack methodologies devised for graph-based models—encompassing both poisoning (training-time) and evasion (test-time) attacks—against two representative instantiations, LLAGA (Chen et al. 2024) and GRAPH-PROMPTER (Liu et al. 2024). Beyond this transferability analysis, we uncover a previously unrecognized attack surface specific to LLAGA, wherein an adversary can inject malicious nodes as placeholders within the node sequence template so as to precipitate a severe deterioration in predictive performance. Our systematic analysis demonstrates that particular design choices governing graph encoding can, counterintuitively, amplify attack efficacy, yielding three principal findings: (1) the node sequence template of LLAGA heightens its susceptibility; (2) the GNN encoder underpinning GRAPH-PROMPTER exhibits comparatively greater resilience; and (3) both paradigms remain acutely vulnerable to imperceptible feature perturbation attacks. Finally, we advance an end-to-end defense framework, GALGUARD, that couples an LLM-based feature correction module—devised to attenuate feature-level perturbations—with adapted GNN defenses that safeguard against structural manipulation¹

1 Introduction

Graphs—abstractions in which nodes and edges encode entities and the relationships that bind them—furnish a natural formalism for modeling the intricate structures that pervade a spectrum of real-world domains, among them social networks (Fan et al. 2019), biology (Dong, Mücke, and Khosla 2022), finance (Wang et al. 2021), and healthcare (Ahmedt-Aristizabal et al. 2021). Graph Neural Networks (GNNs) (Hamilton, Ying, and Leskovec 2017; Kipf and Welling 2022; Veličković et al. 2018), engineered expressly for graph-structured data, attain state-of-the-art proficiency on tasks such as node classification and link prediction. More recently, the striking success of large language

models (LLMs)—among them GPT-4 (Achiam et al. 2023), Gemini (Team et al. 2023), LLaMA (Touvron et al. 2023; Dubey et al. 2024), Vicuna (Chiang et al. 2023), and PaLM-2 (Anil et al. 2023), all trained on prodigious volumes of textual data—has galvanized interest in their capacity to augment graph-related tasks. This confluence of graph data and LLMs, denoted *graph-aware LLMs* (Xie et al. 2023), has inaugurated a burgeoning line of inquiry in which LLMs are increasingly enlisted to discharge tasks conventionally entrusted to GNNs (Ye et al. 2023; Fatemi, Halcrow, and Perozzi 2023; Kuang et al. 2023; Zhang et al. 2023; Chen et al. 2023b, 2024). Indeed, a growing body of work demonstrates that graph-aware LLMs can surpass GNNs across a diversity of graph-related tasks (Wenkel, Wolf, and Knyazev 2023; Chen et al. 2023b; Zhu et al. 2024; Chen et al. 2024), thereby establishing them as formidable alternatives.

Notwithstanding their escalating popularity, the susceptibility of graph-aware LLMs to adversarial manipulation remains, to a striking degree, uncharted. Prevailing adversarial attacks on GNNs—spanning both poisoning (training-time) and evasion (test-time) formulations (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Dai et al. 2023)—operate within a black-box regime in which the adversary is denied any knowledge of the model’s internal mechanics. Such attacks subvert *model integrity*, deliberately eroding predictive performance through subtle perturbations of the graph structure or the node features. Whereas the GNN literature has centered its attention overwhelmingly on structural attacks, the fundamentally textual character of graph-aware LLMs renders imperceptible perturbations of node features a consequential yet largely neglected threat.

Impelled by three interlocking observations—the migration from GNNs to LLMs for graph tasks, the black-box provenance of contemporary adversarial attacks on GNNs, and the hitherto unappraised vulnerabilities to feature perturbation—we pose the pivotal question of whether *adversarial attacks conceived for GNNs retain their potency against the nascent graph-aware LLM paradigm*. Compounding this inquiry is the fact that the efficacy of graph-aware LLMs on graph tasks *hinges critically* upon how effectively the graph is assimilated into the LLM. While the processing of node features is comparatively unproblematic, given the LLM’s native facility with textual data, the incorporation of graph structure poses a decidedly more intricate challenge

¹Our code will be released upon acceptance.

that mandates inventive mechanisms for capturing relational information (Chen et al. 2024; Guo et al. 2023; Liu and Wu 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). Accordingly, to address our research question rigorously, we first partition extant graph-aware methods into two classes delineated by their strategy for encoding graphs into LLMs: one that renders graph data as *textual descriptions*, transcribing graph information into natural language, and another that marshals *learned projectors* to reconcile the inherent disparities between the graph and text domains. Because conventional text-based adversarial attacks on LLMs should, in general, suffice to compromise models that ingest textual descriptions of graphs, we train our investigation on learned projector methods, adopting two computationally efficient and representative models, LLAGA (Chen et al. 2024) and GRAPHPROMPTER (Liu et al. 2024), both of which are predicated on a frozen pre-trained LLM.

Our analysis reveals that certain design choices governing graph encoding for LLMs can, unexpectedly, magnify the success of attacks. Most notably, we isolate a novel attack surface in LLAGA that would permit an adversary to precipitate a severe degradation of the model’s performance by injecting malicious nodes as placeholders within the node sequence template. In addition, we advance an end-to-end graph-aware LLM defense, GALGUARD, which weds an LLM-based feature corrector to adapted GNN-based structural defenses in order to furnish robust protection. To our knowledge, we are the first to interrogate the vulnerability of graph-aware LLMs to adversarial attacks. The principal contributions of this paper are summarized as follows:

- We repurpose representative adversarial attacks for GNNs—specifically NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019)—to interrogate the transferability of poisoning and evasion attacks to graph-aware LLMs.
- We construct a taxonomy of graph-aware LLMs organized around their encoding strategies, thereby elucidating how disparate methodologies assimilate graph structures into LLMs.
- We uncover a hitherto unrecognized attack surface in LLAGA, establishing that the injection of malicious nodes as placeholders within the node sequence template can materially degrade its performance.
- We furnish a holistic assessment of the vulnerabilities of graph-aware LLMs by mounting imperceptible attacks on textual node features—a vantage point routinely disregarded in the GNN literature.
- We propose GALGUARD, a novel end-to-end defense strategy that integrates LLM-based feature correction with the adaptation of existing GNN-based structural defenses.

2 Graph Encoding-Based Taxonomy of LLM Adaptations

Graph encoding methods for LLMs may be apportioned into two principal approaches: *textual descriptions of graphs*, in which graph information is encoded as natural language, and *learned graph projectors*, in which a projector is enlisted

to transmute graph data into embeddings or structured representations that LLMs can process directly. We sketch the constituent encoding approaches of each category in turn. Figure 5 furnishes an illustration of both approaches; a full exposition appears in Appendix A.1, and related work on GNN attacks in Appendix A.2.

Graph Representation as a Textual Description. This approach transcribes graph data into natural language, thereby enabling LLMs to process it as textual descriptions, frequently accompanied by explicit instructions with which to query the LLM (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a; Zhang 2023). In multimodal settings, Liu et al. (2023) and Su et al. (2022) conjoined molecule graphs with textual descriptions. Extending this thread, Huang et al. (2023a) scrutinized the incorporation of graph structure into LLM prompts; their findings intimate that LLMs are prone to processing graph-related prompts as contextual paragraphs, a tendency that accentuates the salience of prompt design, as evidenced in studies such as (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024; He et al. 2024). A parallel body of research has concentrated on marshaling LLMs to enrich node features and graph structures, to serve as predictors, or to act as feature extractors for GNNs (Chen et al. 2023a; Chandra et al. 2020; Chen et al. 2023b; Mavromatis et al. 2023). On the inference front, Zhu et al. (2024) devised a fine-tuning method for textual graphs that fuses LLMs with GNNs, while Duan et al. (2023) introduced an effective approach that augments textual graph learning. Although these methods afford a straightforward avenue for deploying LLMs on graph tasks, they frequently falter in capturing structural intricacies—particularly for large graphs—and remain heavily contingent upon prompt engineering.

Graph Representation as a Learned Projector. To surmount the limitations of textual representations, learned projector methods transmute graph data into embeddings or graph vectors amenable to more direct processing by LLMs. This class encompasses methods such as LLAGA, which couples node-level templates with a linear projector (Chen et al. 2024), as well as others that graft GNN layers onto a pre-trained LLM (Liu et al. 2024; Qin et al. 2023). Still further approaches deploy graph neural prompts to encode knowledge graphs (Tian et al. 2024), engineer prompt-based node feature extractors equipped with a GNN adapter (Huang et al. 2023b), or distill graph information into fixed-length prefixes by means of graph transformers (Chai et al. 2023; Yang et al. 2021; Peng et al. 2024).

This Work: We concentrate on projector methods predicated on *frozen pre-trained LLMs*, since they are computationally economical and obviate the substantial expenditure entailed by retraining (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). For our investigation, we adopt two representative models: LLAGA (Chen et al. 2024), which marries node-level templates to a linear projector, and GRAPHPROMPTER (Liu et al. 2024), which pairs a GNN with a linear projector. We set aside textual description methods, inasmuch as they frequently founder

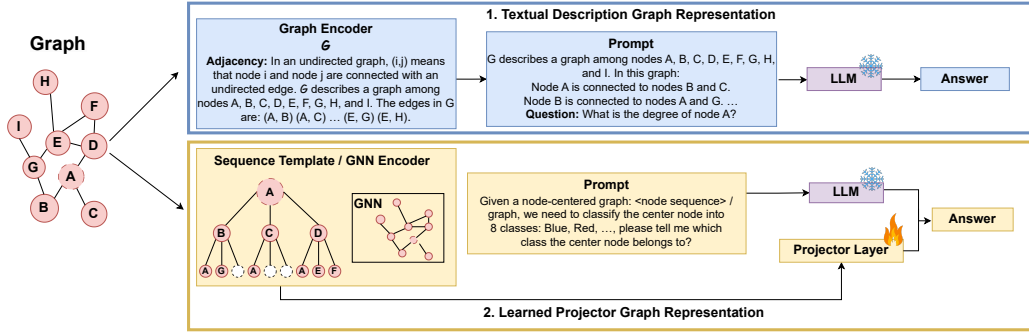


Figure 1: **Graph encoding-based adaptations of LLMs.** In the textual description graph representation, graphs are explained textually (top part). In the learned projector representation, graphs are encoded either by node templates that turns the graph into sequences or by GNNs (bottom part). Note that in both cases, the LLM is frozen. Only the projector that maps the graph into tokens is trained.

on scalability and do not engender the novel, graph-specific vulnerabilities that constitute the object of our investigation.

3 Threat Model

We formulate our threat model on the foundation of established research into adversarial attacks on graphs, and we extend its purview to graph-aware LLMs (Zügner, Akbarnejad, and Günnemann 2018; Wu et al. 2019; Zügner and Günnemann 2019; Ma, Ding, and Mei 2020; Jin et al. 2021; Dai et al. 2023; Li et al. 2023). In social networks, for instance, adversaries typically command no more than a handful of bot accounts; this circumscribed access dovetails with our threat model, in which the adversary manipulates a small subset of nodes so as to elude detection or to sway the classification of legitimate users. A cognate scenario arises on Wikipedia, where hoax articles that maintain sparse connections to legitimate ones can be strategically linked in order to distort the classification of genuine content, thereby inducing the misclassification of real articles’ categories. Such transferability is especially disquieting given that these advanced models are poised for deployment in high-stakes applications—recommendation systems, social media analysis, and even healthcare—where they may ultimately supplant GNNs. Accordingly, in keeping with prior work on adversarial attacks in graphs, we characterize the threat model along three dimensions—model access, adversarial capabilities, and attack strategies—before articulating the attacker’s goal.

Model Access. The attacker operates within a black-box regime, bereft of any knowledge of the model’s architecture or parameters, and can influence the model only indirectly, through interaction with the training or test data.

Adversarial Capabilities. Consonant with existing poisoning (evasion) attacks, the attacker enjoys complete access to the training (inference) data of the target model. Constrained by a predefined budget and subject to unnoticeability requirements—which are intended to forestall detection—the attacker may modify or inject data.

Attack Strategies. The attacker marshals two principal strategies: feature manipulation, which alters node features, and structure manipulation, which adds or removes edges.

Attacker’s Goal. The attacker’s objective is to compromise model integrity by degrading overall performance—concretely, by inflating the misclassification rate of the classification model.

4 Experimental Setup

Dataset. We avail ourselves of three widely adopted graph datasets—Cora, Citeseer, and PubMed (Yang, Cohen, and Salakhudinov 2016). These text-attributed graph datasets, which span small-to-medium scales in both size and sparsity, constitute customary benchmarks for models of the kind examined in our study (Chen et al. 2023a, 2024; Liu et al. 2024). To further corroborate scalability, we additionally incorporate the substantially larger ArXiv dataset (Hu et al. 2020) in a supplementary experiment (see Appendix A.5). We adhere to the standard train/validation/test partitions of 6:2:2 for the three smaller datasets and 5:2:3 for ArXiv, in accordance with (Chen et al. 2024). Detailed statistics are reported in Table 1.

Table 1: Summary statistics of the datasets employed in our evaluation.

Data	#Node	#Edge	Sparsity (%)
Cora	2708	5429	14.81
CiteSeer	3327	4614	8.34
PubMed	19717	44338	2.28

Tasks. We designate *node classification* as our primary task—a canonical instrument for the evaluation of graph machine learning models and the prevailing setting for adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Liu and Wu 2023).

Evaluation Metrics. We adopt *accuracy*—the proportion of correctly predicted labels—as our evaluation metric. Each experiment is replicated three times, and we report the resulting mean together with its standard deviation. All experiments were executed on a machine outfitted with an NVIDIA A40 GPU furnishing 48GB of memory.

Table 2: Performance of NETTACK and METAATTACK across the Cora, Citeseer, and PubMed datasets for the graph-aware LLMs LLAGA and GRAPHPROMPTER.

Attack	Type	Cora		Citeseer		PubMed	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Clean	—	0.89 $\pm$ 0.07	0.60 $\pm$ 0.03	0.71 $\pm$ 0.04	0.70 $\pm$ 0.03	0.90 $\pm$ 0.04	0.90 $\pm$ 0.02
NETTACK	Poisoning	0.87 $\pm$ 0.04 (2%)	0.60 $\pm$ 0.03 (0%)	0.64 $\pm$ 0.05 (10%)	0.68 $\pm$ 0.02 (3%)	0.89 $\pm$ 0.03 (1%)	0.88 $\pm$ 0.02 (2%)
	Evasion	0.55 $\pm$ 0.09 (38%)	0.53 $\pm$ 0.05 (12%)	0.59 $\pm$ 0.04 (19%)	0.61 $\pm$ 0.03 (14%)	0.84 $\pm$ 0.05 (7%)	0.81 $\pm$ 0.02 (10%)
METAATTACK	Poisoning	0.79 $\pm$ 0.03 (11%)	0.58 $\pm$ 0.05 (3%)	0.63 $\pm$ 0.04 (12%)	0.65 $\pm$ 0.06 (7%)	0.89 $\pm$ 0.08 (1%)	0.85 $\pm$ 0.02 (6%)
	Evasion	0.44 $\pm$ 0.06 (51%)	0.52 $\pm$ 0.04 (13%)	0.50 $\pm$ 0.03 (35%)	0.56 $\pm$ 0.02 (22%)	0.73 $\pm$ 0.04 (19%)	0.77 $\pm$ 0.03 (14%)

Choice of Graph-aware LLM and Adversarial Attack Methods. We now expound our rationale for the graph-aware LLM methods and the adversarial attack methods marshaled to interrogate the vulnerabilities of graph-aware LLMs.

Graph-Aware LLM Methods. We enlist two representative graph-aware LLM methods: LLAGA (Chen et al. 2024) and GRAPHPROMPTER (Liu et al. 2024). Both are predicated on a *frozen pre-trained LLM*, a design that renders them computationally economical and readily adaptable to diverse graph tasks and LLM architectures. LLAGA couples node-level templates with a linear projector, whereas GRAPHPROMPTER pairs a GNN-based encoder with a linear projector; in either case, the projector is instantiated as a 2-layer multi-layer perceptron (MLP). The foundational LLMs underpinning LLAGA and GRAPHPROMPTER are, respectively, Vicuna-7B (Chiang et al. 2023) and LLAMA2-7B (Touvron et al. 2023).

Adversarial Attack Methods. To conduct our investigation, we deploy the poisoning and evasion attacks furnished by two representative adversarial attack methods, NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019). NETTACK is a targeted attack that seeks the minimal perturbations sufficient to misclassify particular nodes, whereas METAATTACK mounts untargeted attacks by treating the graph structure itself as a hyperparameter to be optimized. For NETTACK, we follow the protocol of (Li et al. 2023; Zügner and Günnemann 2019), sequentially targeting each node and aggregating the outcomes, thereby effectively recasting it as an untargeted evaluation. Throughout all experiments, we perturb no more than 10% of the edges so as to uphold the unnoticeability constraint, in keeping with (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019). Full details are relegated to Appendix A.2.

5 Results

5.1 Poisoning and Evasion Attacks on Graph-aware LLMs

We analyze poisoning and evasion attacks by adapting the methods of NETTACK (Zügner and Günnemann 2019) and METAATTACK (Zügner and Günnemann 2019). For poisoning attacks on LLAGA, we employ the perturbed graph to generate the node-level sequences and subsequently retrain

the projector; for GRAPHPROMPTER, we leverage the perturbed graph to re-encode it via a GAT and thereafter retrain its projector. For evasion attacks, by contrast, both models are trained on unperturbed data, and perturbations are introduced solely during the inference phase.

Results. Our experiments yield several salient insights, as consolidated in Table 2. First, poisoning attacks exert only a circumscribed impact on graph-aware LLMs, occasioning a maximum performance degradation of 11% across both attack methods—an outcome that intimates that the models’ robust representations, together with their reliance on supplementary context furnished by the LLM, confer a measure of resilience. Evasion attacks, by contrast, prove markedly efficacious, precipitating performance degradation as severe as 51%. Second, among the attack methods, METAATTACK generally eclipses NETTACK for both poisoning and evasion across all datasets—a disparity attributable to METAATTACK’s capacity to craft small yet highly effective perturbations by treating graph edges as hyperparameters. Finally, we discern that denser datasets are more susceptible to attack than their sparser counterparts: LLAGA, for instance, sustained a 51% performance decrement on the denser Cora under an evasion attack, yet only a 19% decrement on the sparser PubMed dataset.

5.2 Which Graph-aware Paradigm is More Susceptible to Graph Adversarial Attacks and Why?

Our experiments (Table 2) establish that LLAGA is more susceptible to attack than GRAPHPROMPTER, a disparity rooted in its pronounced *sensitivity to structural perturbations*. LLAGA assimilates the graph structure through two complementary mechanisms: the generation of a fixed-length node-level sequence for each node and the computation of a graph Laplacian embedding. Because every position within this sequence corresponds uniquely to a relative structural position in the original graph, and because the node embeddings are derived directly from the perturbed graph structure, such attacks materially contaminate the information conveyed to the LLM, thereby precipitating performance degradation. In stark contrast, GRAPHPROMPTER’s textual node embeddings persist unaltered under structural perturbation, since only the GNN component is affected; this design permits the unperturbed textual features to buttress the perturbed graph structure, thereby fortifying robustness.

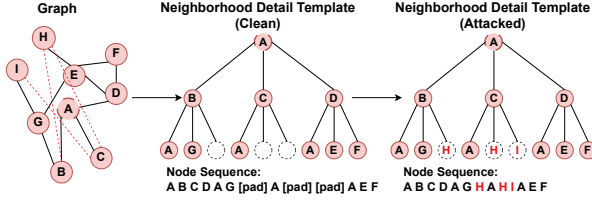


Figure 2: Illustration of the node sequence template injection attacks. Depicted here is the Non-adjacent Injection (NI) attack, in which the red dashed lines denote the edge perturbations introduced by the attacker. In the Supernode Injection (SI) and Multiple Supernode Injection (MSI) attacks, the supernode(s) are instead inserted in place of the placeholders. Here, $k = 3$.

6 Node Sequence Template Injection Attacks

We expose a novel attack surface in LLAGA—one that arises specifically from exploiting the neighborhood detail template through which the node sequences are constructed. The attack is illustrated in Figure 2, and the accompanying algorithm is furnished in Appendix A.4. To derive the node sequence for a given node u , LLAGA’s neighborhood detail template first erects a *fixed-shape* computational tree for each node, in which the children of a node are its sampled 1-hop neighbors; this procedure is then reiterated to sample neighbors at each successive level of the tree. Should a node possess an insufficient number of neighbors, a placeholder is interposed so as to preserve the fixed cardinality of, for example, $k = 10$ children, in fidelity to LLAGA’s original design (see “Clean” in Figure 2 and lines 1–14 of the algorithm). It is precisely this rigid, fixed structure that opens a new attack surface.

We demonstrate this new attack surface in LLAGA by exploiting its neighborhood detail template algorithm, whereby an attacker can artfully inject malicious nodes into placeholder positions through graph manipulation and thereby substantially degrade model performance. We propose three attack strategies to this end. The first, which we term *Non-Adjacent Injection (NI)*, injects nodes lying outside the 2-hop neighborhood of a target node u (see “Attacked” in Figure 2 and lines 21–27). The second, denoted *Supernode Injection (SI)*, injects a single high-degree node as a placeholder, taking care that it too resides beyond the target node’s 2-hop neighborhood. The third, *Multiple Supernode Injection (MSI)*, constitutes a variant of *SI* that injects multiple distinct supernodes as placeholders (lines 35–43). The underlying rationale is that the intrusion of uninformative nodes into the node sequence corrupts the representation of the target node, engendering a poorly trained projection layer and, in turn, compromising the integrity of the model’s learning and inference processes.

Results. As reported in Table 3, LLAGA exhibits pronounced resilience to poisoning attacks on the Cora and Citeseer datasets, yet succumbs to a substantial performance decline under evasion, its accuracy descending to 0.42, 0.36, and 0.30 on Cora and to 0.36, 0.30, and 0.24 on Citeseer for the *NI*, *SI*, and *MSI* attacks, respectively.

Table 3: Results of the node sequence template injection attack across the various datasets and methods. *NI*, *SI*, and *MSI* denote, respectively, the non-adjacent, single-supernode, and multiple-supernode injection attacks. Clean designates the performance of the model in the absence of any attack.

Data	Attack	NI	SI	MSI
Cora	Clean	—	0.89 ± 0.07	—
	Poisoning	0.86 ± 0.03	0.84 ± 0.06	0.82 ± 0.04
	Evasion	0.42 ± 0.08	0.36 ± 0.05	0.30 ± 0.05
Citeseer	Clean	—	0.71 ± 0.04	—
	Poisoning	0.69 ± 0.05	0.66 ± 0.03	0.62 ± 0.02
	Evasion	0.36 ± 0.06	0.30 ± 0.04	0.24 ± 0.05
PubMed	Clean	—	0.90 ± 0.04	—
	Poisoning	0.88 ± 0.04	0.85 ± 0.05	0.81 ± 0.03
	Evasion	0.85 ± 0.07	0.82 ± 0.04	0.76 ± 0.03

Conversely, on the considerably larger PubMed dataset, the model manifests markedly greater resilience to all injection attacks, incurring only a modest reduction in accuracy under evasion (0.85, 0.82, and 0.76). One explanation is that, in smaller and more tightly interconnected graphs such as Cora and Citeseer, a localized injection exerts a more concentrated influence, gravely disrupting the model’s capacity to reason over the local neighborhood; on the far more expansive PubMed graph, by contrast, the effect of an identical attack is diluted, occasioning a comparatively negligible degradation in performance.

Relative to traditional graph adversarial methods, node injection attacks prove decidedly more effective. The *MSI* method, for instance, attained evasion accuracies of 0.30, 0.24, and 0.76 on Cora, CiteSeer, and PubMed, respectively, whereas METAATTACK managed only 0.44, 0.50, and 0.73 (Table 2).

7 Poisoning and Evasion Attack via Feature Perturbation

Existing graph adversarial attacks concentrate predominantly on structural perturbations (Li et al. 2023; Wu et al. 2019; Dai et al. 2018; Sun et al. 2022; Zügner and Günnemann 2019), the sole exception being NETTACK (Zügner, Akbarnejad, and Günnemann 2018), which does not transfer to the textual domain. Given that structural attacks achieve only moderate success against robust models such as GRAPH PROMPTER, and that textual features have proven pivotal to model robustness, we turn to unnoticeable feature perturbation attacks as an alternative means of degrading model performance.

We adopt the imperceptible attack strategies advanced by (Boucher et al. 2022), albeit with one consequential modification: we employ a task-agnostic translation objective (English to French) to generate the perturbations. The aim is to alter the input text such that the modifications are visually indistinguishable yet nonetheless induce an erroneous translation, thereby corrupting the encoded semantics—the underlying intuition being that a text whose translation is rendered incorrect must likewise harbor corrupted encoded semantics. We instantiate two imperceptible attacks: *Homoglyphs*, which

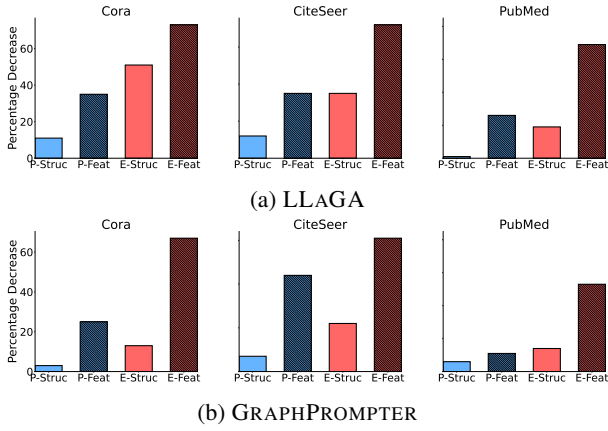


Figure 3: Performance of structure- and feature-based adversarial attacks on graph-aware LLMs. Here, attack success is quantified as the percentage decrease (y-axis) relative to the original accuracy. P-Struc, P-Feat, E-Struc, and E-Feat denote, respectively, the poisoning and evasion attacks grounded in structural and feature perturbation.

substitute characters with visually near-identical counterparts, and *Reorderings*, which invoke directional control characters to perturb the rendering sequence. Examples of Homoglyphs and Reorderings are provided in Appendix A.6.

Following (Boucher et al. 2022), we harness differential evolution (Storn and Price 1997) to optimize these perturbations, maximizing the Levenshtein distance between the model’s outputs on the perturbed and unperturbed text. The perturbation budget is fixed at 10% of the average length of each textual feature.

Results. As reported in Table 4, Reordering attacks are, across all models and datasets, generally more potent than their Homoglyph counterparts. On Cora and CiteSeer, LLAGA betrays vulnerability to both attack types: poisoning attacks precipitate accuracy reductions of 27% and 23% under Homoglyph and a more pronounced 35% and 25% under Reordering, respectively. In both instances, GRAPH-PROMPTER proves the more resilient of the two. The PubMed dataset displays analogous tendencies, though the aggregate impact remains less pronounced than on the smaller datasets.

A juxtaposition with structural perturbations lays bare a fundamental reconfiguration of vulnerabilities, as depicted in Figure 3. Feature perturbation attacks—Reordering in particular—outstrip structural attacks (METAATTACK) by a substantial margin on both LLAGA and GRAPH-PROMPTER. This finding intimates that graph-aware LLMs accord greater weight to input features than to graph structure, rendering them disproportionately vulnerable to feature perturbations. We probe this phenomenon further by amalgamating both attack types within a unified setting in the following section.

8 A Unified Imperceptible Attack on Graph-aware LLMs

We propose a unified attack that conjoins feature and structural perturbations so as to constitute a more imperceptible

yet more effective adversarial strategy. We hypothesize that coupling these two attack modalities will degrade model accuracy more severely than either in isolation, all the while confining the alterations to the graph structure and features to imperceptible magnitudes.

For the structural component, we employ METAATTACK, which recasts the graph structure through subtle adjustments to its edges; for the feature component, we invoke a conjunction of Homoglyph and Reordering attacks, as delineated in the preceding section. In concert, these perturbations engender a highly imperceptible attack that assails, simultaneously, both the relational and the textual information upon which graph-aware LLMs depend.

Results. As reported in Table 5, the unified attack—which jointly administers structural and feature perturbations—proves markedly more effective than either perturbation method in isolation at degrading model performance across the various datasets. This analysis affords a pivotal insight into the architecture of graph-aware LLMs: unlike traditional GNNs, which are impaired principally by structural perturbations, these models evince substantial vulnerability to feature-level attacks. The efficacy of the unified attack betokens their reliance on the mutual consistency of graph structure and node features, a dependence that renders them acutely susceptible to such combined and inconspicuous modifications.

9 Ablation Study

9.1 Varying Feature Perturbation

To sharpen our understanding of the relationship between perturbation level and attack success, we examine the consequences of varying the proportion of perturbed features. Our analysis of the Reordering attack—which mirrors the patterns exhibited by the Homoglyph attack—discloses a consistent trend as the perturbation proportion ranges from 1% to 10%. For every model and dataset (Cora, CiteSeer, and PubMed), diminishing the fraction of perturbed features yields a correspondingly less successful attack, as one would anticipate. Notably, as the perturbation level recedes, the success rates of the poisoning and evasion attacks tend to converge. The explanation is that, with fewer perturbations, the adversarial signal grows too attenuated to appreciably perturb the model, irrespective of whether it was introduced during the training (poisoning) or the inference (evasion) phase. A detailed plot illustrating these trends is deferred to the Appendix (Figure 2).

9.2 Does the Choice of LLM Affect Attack Performance?

We assess whether the foundation LLM exerts any influence on the adversarial robustness of graph-aware LLMs. LLAGA is built upon Vicuna-7B (Chiang et al. 2023) and GRAPH-PROMPTER upon Llama2-7B (Touvron et al. 2023). We supplant LLAGA’s backbone with Llama2-7B and juxtapose the resulting performance under poisoning and evasion attacks across all datasets. Both configurations attain comparable accuracy in the clean setting (e.g., Vicuna: 0.89 ± 0.07 ; Llama2: 0.90 ± 0.03 on Cora) and manifest commensurate

Table 4: Performance under feature perturbation attacks, with the percentage decrement from clean accuracy given in parentheses.

Data	Method	Poisoning		Evasion	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Cora	Homoglyph	0.65 $\pm$ 0.05 (27%)	0.52 $\pm$ 0.04 (13%)	0.41 $\pm$ 0.04 (54%)	0.38 $\pm$ 0.03 (37%)
	Reordering	0.58 $\pm$ 0.06 (35%)	0.45 $\pm$ 0.05 (25%)	0.24 $\pm$ 0.05 (73%)	0.20 $\pm$ 0.03 (67%)
Citeseer	Homoglyph	0.54 $\pm$ 0.04 (23%)	0.65 $\pm$ 0.04 (7%)	0.32 $\pm$ 0.03 (55%)	0.52 $\pm$ 0.05 (26%)
	Reordering	0.46 $\pm$ 0.03 (35%)	0.39 $\pm$ 0.02 (44%)	0.20 $\pm$ 0.05 (72%)	0.27 $\pm$ 0.02 (61%)
PubMed	Homoglyph	0.72 $\pm$ 0.04 (20%)	0.83 $\pm$ 0.04 (8%)	0.36 $\pm$ 0.06 (60%)	0.55 $\pm$ 0.03 (39%)
	Reordering	0.67 $\pm$ 0.05 (26%)	0.80 $\pm$ 0.03 (11%)	0.28 $\pm$ 0.04 (69%)	0.42 $\pm$ 0.04 (53%)

Table 5: Model accuracy under the unified imperceptible attack, with the percentage decrement from clean accuracy reported in parentheses.

Data	Attack	LLAGA	GRAPHPROMPTER
Cora	Poisoning	0.35 $\pm$ 0.05 (61%)	0.28 $\pm$ 0.08 (53%)
	Evasion	0.14 $\pm$ 0.06 (84%)	0.11 $\pm$ 0.05 (82%)
Citeseer	Poisoning	0.26 $\pm$ 0.04 (63%)	0.22 $\pm$ 0.03 (69%)
	Evasion	0.10 $\pm$ 0.09 (86%)	0.13 $\pm$ 0.07 (81%)
PubMed	Poisoning	0.39 $\pm$ 0.07 (57%)	0.32 $\pm$ 0.04 (64%)
	Evasion	0.12 $\pm$ 0.05 (87%)	0.19 $\pm$ 0.04 (79%)

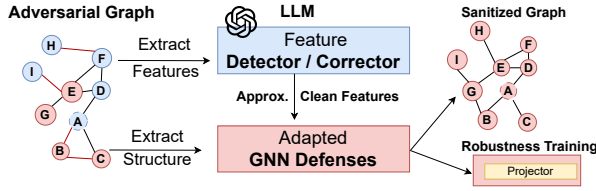


Figure 4: **Overview of GALGUARD defense.** Perturbed node features and edges are rendered in blue and red, respectively. The LLM detects perturbations in the node features extracted from an adversarial graph and synthesizes a corrected rendition. The refined features, in conjunction with the perturbed edges, are subsequently routed through existing GNN defenses, which may be employed either to obtain a sanitized graph or, alternatively, adapted for the robustness training of the projectors.

vulnerability under attack, and the same tendency persists when the LLMs are interchanged within GRAPHPROMPTER. These results intimate that the choice of foundation LLM bears only a negligible influence on adversarial robustness. The full results are tabulated in Table 2 of the Appendix.

10 Defense: GALGUARD

Existing GNN defenses against adversarial attacks prove inadequate for graph-aware LLMs, owing to the latter’s dual susceptibility to both structural and feature perturbations. A great many defenses lean on node feature similarity to prune or reweight edges (Wu et al. 2019; Dai et al. 2018; Zhang and Zitnik 2020; Jin et al. 2020; Zhang and Ma 2024), yet they presuppose access to pristine features; once those features are themselves perturbed, such methods founder (see Figure 3 in the Appendix). By the same token, defenses that decouple structure from features (Shanthamallu, Thiagarajan, and Spanias 2021; Wu et al. 2021), and even OCR-based sanitization (Boucher et al. 2022), disintegrate under coordinated attack. This dual vulnerability compels the design of a new, integrated defense.

To this end, we propose graph-aware LLM defense (GALGUARD), an end-to-end defense strategy that fuses two complementary components: (1) an LLM-based feature corrector and (2) adapted GNN-based structural defenses.

Feature Robustness Strategy. As illustrated in Figure 4, an LLM functions as an adaptive mechanism that scrutinizes the textual node features for anomalous patterns, furnishing approximate corrections with which to restore feature integrity and, in so doing, attenuating the effect of feature-level perturbations. This LLM-based correction constitutes a pragmatic remedy for feature vulnerabilities; the attendant instructions and prompts are consigned to Appendix A.6. For our experiments, we adopt GPT-4 Turbo as the LLM corrector.

Graph Structure Robustness Strategy. To bolster structural robustness, we pursue a two-stage approach. First, we adopt a *graph purification heuristic* (Wu et al. 2019) as a preprocessing step: for each node v , we compute the cosine similarity between its embedding $\phi(x_v)$ and the embeddings $\phi(x_u)$ of its neighbors, and we excise edges whose similarity falls below a threshold. Second, we integrate *GNNGuard* (Zhang and Zitnik 2020), applying its principles directly to the message-passing layers of GRAPHPROMPTER. For LLAGA, we devise a novel adaptation whereby the purification step is woven into the construction of the computational tree underlying its node sequence. Furthermore, we introduce a learnable *global structural context embedding* (M_{global}) as a trainable parameter within LLAGA’s projector; by concatenating M_{global} with the purified node sequence embedding and training the two jointly, the model learns to operate robustly on potentially perturbed graph inputs. The plug-and-play design of GALGUARD further permits alternative defenses—for instance, Pro-GNN (Jin et al. 2020)—to be substituted for GNNGuard.

10.1 Results

Table 6 summarizes the performance, measured in accuracy, of the models under METAATTACK, alongside our two defense variants: GALGUARD_p, which conjoins a feature corrector with a graph purification module, and the full GALGUARD, which integrates the feature corrector, the graph purification module, and GNNGuard (or its adaptation). Across every evaluated dataset (Cora, CiteSeer, and PubMed) and for both the poisoning and the evasion regimes, our defenses consistently temper the impact of METAATTACK. In particular, GALGUARD_p proves an effective foundational step, attesting that redressing feature perturbations with the LLM corrector and applying an initial round of graph purification substantially enhance robustness. Notably, the full GALGUARD con-

sistently confers superior gains in model robustness relative to GALGUARD_p, thereby throwing into relief the synergistic dividend accruing from the integration of the GNN-inspired structural defense.

Table 6: Performance of METAATTACK and the proposed defense GALGUARD under poisoning and evasion attacks. GALGUARD_p conjoins feature correction with graph purification.

Data Method		LLAGA		GRAPHPROMPTER	
		Poisoning	Evasion	Poisoning	Evasion
Cora	METAATTACK	0.79 $\pm$ 0.03	0.44 $\pm$ 0.06	0.58 $\pm$ 0.05	0.52 $\pm$ 0.04
	GALGUARD _p	0.82 $\pm$ 0.05	0.62 $\pm$ 0.04	0.59 $\pm$ 0.03	0.56 $\pm$ 0.04
	GALGUARD	0.85$\pm$0.04	0.83$\pm$0.04	0.60$\pm$0.05	0.59$\pm$0.03
CiteSeer	MetaAttack	0.63 $\pm$ 0.04	0.50 $\pm$ 0.03	0.65 $\pm$ 0.06	0.56 $\pm$ 0.03
	GALGUARD _p	0.64 $\pm$ 0.03	0.56 $\pm$ 0.02	0.67 $\pm$ 0.05	0.60 $\pm$ 0.06
	GALGUARD	0.67$\pm$0.04	0.62$\pm$0.04	0.68$\pm$0.04	0.64$\pm$0.05
PubMed	MetaAttack	0.89 $\pm$ 0.08	0.73 $\pm$ 0.04	0.85 $\pm$ 0.02	0.77 $\pm$ 0.03
	GALGUARD _p	0.90 $\pm$ 0.06	0.78 $\pm$ 0.03	0.85 $\pm$ 0.04	0.81 $\pm$ 0.04
	GALGUARD	0.90$\pm$0.01	0.87$\pm$0.02	0.89$\pm$0.02	0.89$\pm$0.01

11 Conclusion

In this paper, we take a substantial first step toward exposing the vulnerabilities of graph-aware LLMs to adversarial attacks—a domain that has hitherto remained largely uncharted. Our work reveals that the very mechanisms enlisted to integrate graph data constitute a critical wellspring of vulnerability. We uncovered new attack surfaces, demonstrating that design choices such as the node sequence template in LLAGA appreciably amplify a model’s susceptibility. Our findings further establish that feature perturbation attacks are strikingly effective—in contradistinction to their impotence against traditional GNNs—frequently eclipsing conventional structural attacks, and that a unified attack conjoining both perturbation modalities can reduce graph-aware LLMs to near-uselessness.

To counter these threats, we propose an end-to-end defense, GALGUARD, which marries an LLM-based feature corrector to adapted GNN-based structural defenses in order to furnish robust protection. As these models grow ever more pervasive and hold increasing promise for applications spanning social networks, healthcare, and finance, securing their resilience against adversarial exploitation becomes an imperative of the first order. We hope this work will serve as a foundation for subsequent inquiry—galvanizing deeper exploration of the vulnerabilities and defenses of graph-aware LLMs, and cultivating alike the theoretical insights and the practical safeguards indispensable to their real-world deployment.

References

Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.

Ahmedt-Aristizabal, D.; Armin, M. A.; Denman, S.; Fookes, C.; and Petersson, L. 2021. Graph-based deep learning for medical diagnosis and analysis: past, present and future. *Sensors*, 21(14): 4758.

Anil, R.; Dai, A. M.; Firat, O.; Johnson, M.; Lepikhin, D.; Passos, A.; Shakeri, S.; Taropa, E.; Bailey, P.; Chen, Z.; et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*.

Bi, B.; Liu, S.; Wang, Y.; Mei, L.; and Chen, X. 2024. Scalable Link Prediction on Large-Scale Heterogeneous Graphs with Large Language Models. *arXiv preprint arXiv:2401.13227*.

Boucher, N.; Shumailov, I.; Anderson, R.; and Papernot, N. 2022. Bad characters: Imperceptible nlp attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*, 1987–2004. IEEE.

Brannon, W.; Fulay, S.; Jiang, H.; Kang, W.; Roy, B.; Kabbara, J.; and Roy, D. 2023. Congrat: Self-supervised contrastive pretraining for joint graph and text embeddings. *arXiv preprint arXiv:2305.14321*.

Chai, Z.; Zhang, T.; Wu, L.; Han, K.; Hu, X.; Huang, X.; and Yang, Y. 2023. GraphLLM: Boosting Graph Reasoning Ability of Large Language Model. *arXiv preprint arXiv:2310.05845*.

Chandra, S.; Mishra, P.; Yannakoudakis, H.; Nimishakavi, M.; Saeidi, M.; and Shutova, E. 2020. Graph-based Modeling of Online Communities for Fake News Detection. *arXiv preprint arXiv:2008.06274*.

Chen, R.; Zhao, T.; Jaiswal, A.; Shah, N.; and Wang, Z. 2024. LLaGA: Large Language and Graph Assistant. In *Forty-first International Conference on Machine Learning*.

Chen, Z.; Mao, H.; Li, H.; Jin, W.; Wen, H.; Wei, X.; Wang, S.; Yin, D.; Fan, W.; Liu, H.; and Tang, J. 2023a. Exploring the Potential of Large Language Models (LLMs) in Learning on Graphs. *arXiv preprint arXiv:2307.03393*.

Chen, Z.; Mao, H.; Wen, H.; Han, H.; Jin, W.; Zhang, H.; Liu, H.; and Tang, J. 2023b. Label-free Node Classification on Graphs with Large Language Models (LLMs). *arXiv preprint arXiv:2310.04668*.

Chiang, W.-L.; Li, Z.; Lin, Z.; Sheng, Y.; Wu, Z.; Zhang, H.; Zheng, L.; Zhuang, S.; Zhuang, Y.; Gonzalez, J. E.; Stoica, I.; and Xing, E. P. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality.

Conti, M.; Li, J.; Picek, S.; and Xu, J. 2022. Label-only membership inference attack against node-level graph neural networks. In *Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security*, 1–12.

Dai, E.; Lin, M.; Zhang, X.; and Wang, S. 2023. Unnoticeable backdoor attacks on graph neural networks. In *Proceedings of the ACM Web Conference 2023*, 2263–2273.

Dai, H.; Li, H.; Tian, T.; Huang, X.; Wang, L.; Zhu, J.; and Song, L. 2018. Adversarial attack on graph structured data. In *International conference on machine learning*, 1115–1124. PMLR.

- Dong, N.; Mücke, S.; and Khosla, M. 2022. Mucomid: A multitask graph convolutional learning framework for miRNA-disease association prediction. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 19(6): 3081–3092.
- Duan, K.; Liu, Q.; Chua, T.-S.; Yan, S.; Ooi, W. T.; Xie, Q.; and He, J. 2023. SimTeG: A Frustratingly Simple Approach Improves Textual Graph Learning. *arXiv preprint arXiv:2308.02565*.
- Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Duddu, V.; Boutet, A.; and Shejwalkar, V. 2020. Quantifying privacy leakage in graph embedding. In *MobiQuitous 2020-17th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*, 76–85.
- Fan, W.; Ma, Y.; Li, Q.; He, Y.; Zhao, E.; Tang, J.; and Yin, D. 2019. Graph neural networks for social recommendation. In *The world wide web conference*, 417–426.
- Fatemi, B.; Halcrow, J.; and Perozzi, B. 2023. Talk like a Graph: Encoding Graphs for Large Language Models. *arXiv preprint arXiv:2310.04560*.
- Gong, N. Z.; and Liu, B. 2018. Attribute inference attacks in online social networks. *ACM Transactions on Privacy and Security (TOPS)*, 21(1): 1–30.
- Guo, J.; Du, L.; Liu, H.; Zhou, M.; He, X.; and Han, S. 2023. GPT4Graph: Can Large Language Models Understand Graph Structured Data ? An Empirical Evaluation and Benchmarking. *arXiv preprint arXiv:2305.15066*.
- Hamilton, W.; Ying, Z.; and Leskovec, J. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30.
- He, X.; Bresson, X.; Laurent, T.; Perold, A.; LeCun, Y.; and Hooi, B. 2024. Harnessing Explanations: LLM-to-LM Interpreter for Enhanced Text-Attributed Graph Representation Learning. In *The Twelfth International Conference on Learning Representations*.
- He, X.; Wen, R.; Wu, Y.; Backes, M.; Shen, Y.; and Zhang, Y. 2021. Node-level membership inference attacks against graph neural networks. *arXiv preprint arXiv:2102.05429*.
- Hu, W.; Fey, M.; Zitnik, M.; Dong, Y.; Ren, H.; Liu, B.; Catasta, M.; and Leskovec, J. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33: 22118–22133.
- Huang, J.; Zhang, X.; Mei, Q.; and Ma, J. 2023a. Can LLMs Effectively Leverage Structural Information for Graph Learning: When and Why. *arXiv preprint arXiv:2309.16595*.
- Huang, X.; Han, K.; Bao, D.; Tao, Q.; Zhang, Z.; Yang, Y.; and Zhu, Q. 2023b. Prompt-based Node Feature Extractor for Few-shot Learning on Text-Attributed Graphs. *arXiv preprint arXiv:2309.02848*.
- Jin, W.; Li, Y.; Xu, H.; Wang, Y.; Ji, S.; Aggarwal, C.; and Tang, J. 2021. Adversarial attacks and defenses on graphs. *ACM SIGKDD Explorations Newsletter*, 22(2): 19–34.
- Jin, W.; Ma, Y.; Liu, X.; Tang, X.; Wang, S.; and Tang, J. 2020. Graph structure learning for robust graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 66–74.
- Kipf, T. N.; and Welling, M. 2022. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- Kuang, H.; Xu, J.; Zhang, H.; Zhao, Z.; Zhang, Q.; Huang, X.; and Wei, Z. 2023. Unleashing the Power of Language Models in Text-Attributed Graph. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 8429–8441. Association for Computational Linguistics.
- Li, K.; Liu, Y.; Ao, X.; and He, Q. 2023. Revisiting graph adversarial attack and defense from a data distribution perspective. In *The Eleventh International Conference on Learning Representations*.
- Liu, C.; and Wu, B. 2023. Evaluating Large Language Models on Graphs: Performance Insights and Comparative Analysis. *arXiv preprint arXiv:2308.11224*.
- Liu, S.; Nie, W.; Wang, C.; Lu, J.; Qiao, Z.; Liu, L.; Tang, J.; Xiao, C.; and Anandkumar, A. 2023. Multi-modal molecule structure–text model for text-based retrieval and editing. *Nature Machine Intelligence*, 5(12): 1447–1457.
- Liu, Z.; He, X.; Tian, Y.; and Chawla, N. V. 2024. Can we soft prompt LLMs for graph learning tasks? In *Companion Proceedings of the ACM on Web Conference 2024*, 481–484.
- Ma, J.; Ding, S.; and Mei, Q. 2020. Towards more practical adversarial attacks on graph neural networks. *Advances in neural information processing systems*, 33: 4756–4766.
- Mavromatis, C.; Ioannidis, V. N.; Wang, S.; Zheng, D.; Adeshina, S.; Ma, J.; Zhao, H.; Faloutsos, C.; and Karypis, G. 2023. Train your own GNN teacher: Graph-aware distillation on textual graphs. *arXiv preprint arXiv:2304.10668*.
- Olatunji, I. E.; Hizber, A.; Sihlovec, O.; and Khosla, M. 2023a. Does black-box attribute inference attacks on graph neural networks constitute privacy risk? *arXiv preprint arXiv:2306.00578*.
- Olatunji, I. E.; Nejdil, W.; and Khosla, M. 2021. Membership inference attack on graph neural networks. In *2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 11–20. IEEE.
- Olatunji, I. E.; Rathee, M.; Funke, T.; and Khosla, M. 2023b. Private Graph Extraction via Feature Explanations. *Proceedings on Privacy Enhancing Technologies*.
- Pan, S.; Luo, L.; Wang, Y.; Chen, C.; Wang, J.; and Wu, X. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*.
- Peng, Y.; Lin, S.; Chen, Q.; Xu, L.; Ren, X.; Li, Y.; and Xu, J. 2024. ChatGraph: Chat with Your Graphs. *arXiv preprint arXiv:2401.12672*.
- Qin, Y.; Wang, X.; Zhang, Z.; and Zhu, W. 2023. Disentangled Representation Learning with Large Language Models for Text-Attributed Graphs. *arXiv preprint arXiv:2310.18152*.

- Shanthamallu, U. S.; Thiagarajan, J. J.; and Spanias, A. 2021. Uncertainty-matching graph neural networks to defend against poisoning attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, 9524–9532.
- Storn, R.; and Price, K. 1997. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization*, 11: 341–359.
- Su, B.; Du, D.; Yang, Z.; Zhou, Y.; Li, J.; Rao, A.; Sun, H.; Lu, Z.; and Wen, J.-R. 2022. A molecular multimodal foundation model associating molecule graphs with natural language. *arXiv preprint arXiv:2209.05481*.
- Sun, L.; Dou, Y.; Yang, C.; Zhang, K.; Wang, J.; Philip, S. Y.; He, L.; and Li, B. 2022. Adversarial attack and defense on graph data: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(8): 7693–7711.
- Sun, S.; Ren, Y.; Ma, C.; and Zhang, X. 2023. Large Language Models as Topological Structure Enhancers for Text-Attributed Graphs. *arXiv preprint arXiv:2311.14324*.
- Tan, Y.; Zhou, Z.; Lv, H.; Liu, W.; and Yang, C. 2023. WalkLM: A Uniform Language Model Fine-tuning Framework for Attributed Graph Embedding. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Team, G.; Anil, R.; Borgeaud, S.; Wu, Y.; Alayrac, J.-B.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A. M.; Hauth, A.; et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Tian, Y.; Song, H.; Wang, Z.; Wang, H.; Hu, Z.; Wang, F.; Chawla, N. V.; and Xu, P. 2024. Graph neural prompting with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 19080–19088.
- Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
- Wang, J.; Zhang, S.; Xiao, Y.; and Song, R. 2021. A review on graph neural network methods in financial applications. *arXiv preprint arXiv:2111.15367*.
- Wang, X.; and Wang, W. H. 2022. Group property inference attacks against graph neural networks. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2871–2884.
- Wei, Y.; Fu, S.; Jiang, W.; Kwok, J. T.; and Zhang, Y. 2024a. Rendering graphs for graph reasoning in multimodal large language models. *arXiv preprint arXiv:2402.02130*.
- Wei, Y.; Huang, Q.; Kwok, J. T.; and Zhang, Y. 2024b. Kicgpt: Large language model with knowledge in context for knowledge graph completion. *arXiv preprint arXiv:2402.02389*.
- Wenkel, F.; Wolf, G.; and Knyazev, B. 2023. Pretrained Language Models to Solve Graph Tasks in Natural Language. In *ICML workshop on Structured Probabilistic Inference & Generative Modeling*.
- Wu, H.; Wang, C.; Tyshetskiy, Y.; Docherty, A.; Lu, K.; and Zhu, L. 2019. Adversarial examples on graph data: Deep insights into attack and defense. *arXiv preprint arXiv:1903.01610*.
- Wu, X.; Wu, H.; Zhou, X.; and Lu, K. 2021. CoG: a Two-View Co-training Framework for Defending Adversarial Attacks on Graph. *arXiv preprint arXiv:2109.05558*.
- Xie, H.; Zheng, D.; Ma, J.; Zhang, H.; Ioannidis, V. N.; Song, X.; Ping, Q.; Wang, S.; Yang, C.; Xu, Y.; et al. 2023. Graph-aware language model pre-training on a large graph corpus can help multiple graph applications. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5270–5281.
- Yang, J.; Liu, Z.; Xiao, S.; Li, C.; Lian, D.; Agrawal, S.; Singh, A.; Sun, G.; and Xie, X. 2021. GraphFormers: GNN-nested Transformers for Representation Learning on Textual graph. In *Proceedings of the 35th Conference on Neural Information Processing Systems*.
- Yang, Z.; Cohen, W.; and Salakhudinov, R. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*, 40–48. PMLR.
- Ye, R.; Zhang, C.; Wang, R.; Xu, S.; and Zhang, Y. 2023. Natural Language is All a Graph Needs. *arXiv preprint arXiv:2308.07134*.
- Zhang, A.; and Ma, J. 2024. Defensevgae: Defending against adversarial attacks on graph data via a variational graph autoencoder. In *International Conference on Intelligent Computing*, 313–324. Springer.
- Zhang, J. 2023. Graph-toolformer: To empower llms with graph reasoning ability via prompt augmented by chatgpt. *arXiv preprint arXiv:2304.11116*.
- Zhang, X.; and Zitnik, M. 2020. Gnn-guard: Defending graph neural networks against adversarial attacks. *Advances in neural information processing systems*, 33: 9263–9275.
- Zhang, Z.; Chen, M.; Backes, M.; Shen, Y.; and Zhang, Y. 2022. Inference attacks against graph neural networks. In *31st USENIX Security Symposium (USENIX Security 22)*, 4543–4560.
- Zhang, Z.; Liu, Q.; Huang, Z.; Wang, H.; Lu, C.; Liu, C.; and Chen, E. 2021. Graphmi: Extracting private graph data from graph neural networks. *arXiv preprint arXiv:2106.02820*.
- Zhang, Z.; Wang, X.; Zhang, Z.; Li, H.; Qin, Y.; and Zhu, W. 2023. LLM4DyG: Can Large Language Models Solve Problems on Dynamic Graphs? *arXiv preprint arXiv:2310.17110*.
- Zhao, J.; Zhuo, L.; Shen, Y.; Qu, M.; Liu, K.; Bronstein, M.; Zhu, Z.; and Tang, J. 2023. GraphText: Graph Reasoning in Text Space. *arXiv preprint arXiv:2310.01089*.
- Zhu, Y.; Wang, Y.; Shi, H.; and Tang, S. 2024. Efficient tuning and inference for large language models on textual graphs. *arXiv preprint arXiv:2401.15569*.
- Zügner, D.; Akbarnejad, A.; and Günnemann, S. 2018. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2847–2856.
- Zügner, D.; and Günnemann, S. 2019. Adversarial Attacks on Graph Neural Networks via Meta Learning. In *International Conference on Learning Representations (ICLR)*.

A Appendix

Organization: The Appendix is structured as follows. Appendix A.1 furnishes a detailed taxonomy of graph-aware LLMs. We survey related work on attacks against Graph Neural Networks (GNNs) in Appendix A.2. Appendix A.3 offers an overview of the adversarial attacks NETTACK and METAATTACK, whereas Appendix A.4 elaborates the algorithm underlying the node sequence injection attack on LLAGA. We report the outcomes of our ablation studies in Appendix A.5, encompassing both the effect of varying the quantity of perturbed features and the influence of the base LLM on attack performance. Finally, Appendix A.7 exhibits examples of feature attacks together with the template for the LLM corrector, and Appendix A.8 deliberates upon the broader implications of our research.

Code. Our code will be released upon acceptance.

A.1 Graph Encoding-Based Taxonomy of LLM Adaptations

The integration of large language models (LLMs) with graph data has crystallized into an emergent field of inquiry, wherein a variety of approaches probe how LLMs may be adapted or augmented for graph-related tasks such as reasoning, representation, and inference. These approaches are, by and large, divisible into two categories delineated by their methods of encoding graphs for LLMs: *textual descriptions of graphs*, in which graph information is encoded as natural language, and *learned graph projector*, in which a projector is enlisted to transmute graph data into embeddings or structured forms directly amenable to LLM processing. In this section, we delineate the methods constituting the several encoding approaches for integrating graph data into LLMs. Figure 5 furnishes an illustration of both approaches, and Table 7 provides a synopsis of the distinctions between them.

Graph Representation as a Textual Description In this first category, graph data is rendered as natural language, thereby permitting LLMs to process graph structures as textual descriptions. This approach has been explored at length in works such as (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a), which transcribe graph data into textual formats, frequently paired with explicit instructions with which to query LLMs. For instance, Tan et al. (2023) introduced a unified framework for attributed graph embedding that generates random-walk-based textual sequences from graphs and subsequently employs them to fine-tune LLMs. In a similar spirit, Zhao et al. (2023) addressed graph reasoning within text spaces, translating graph structures into natural language so as to facilitate LLM interaction. Extending this notion, Wei et al. (2024a) enriched textual representations of graphs with visual counterparts, thereby amplifying the reasoning capabilities of LLMs on graph data. In multimodal settings, Liu et al. (2023) and Su et al. (2022) conjoined molecule graphs with textual descriptions, enabling advances in drug discovery and molecular retrieval. Furthermore, Huang et al. (2023a) investigated how the incorporation of graph structure into LLM prompts can

bolster predictive performance; their findings disclosed that LLMs are inclined to process graph-related prompts as contextual paragraphs rather than to recognize graph structures explicitly. This tendency further accentuates the salience of prompt design, as evidenced in studies such as (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024), which extended LLMs to heterogeneous and large-scale graph tasks. Complementing these endeavors, (He et al. 2024) explored the use of LLM-generated explanations as features with which to enhance both the performance and the interpretability of LLMs on graphs.

While still representing graphs as textual descriptions, another line of work interweaves pretraining strategies with GNNs so as to fortify the graph reasoning capabilities of LLMs. Chen et al. (2023a), for example, proposed two modes of LLM-GNN integration: (1) enlisting LLMs to enrich node features for node classification, and (2) employing LLMs as direct predictors for graph tasks. Consonant with this, Chandra et al. (2020) deployed LLMs as text encoders and GNNs as graph structure encoders for tasks such as fake news detection within online communities. Similarly, Chen et al. (2023b) explored the prospect of performing node classification in the absence of explicit labels by having LLMs annotate nodes, which are thereafter used to train a GNN model. In like manner, Mavromatis et al. (2023) pressed an LLM into service as a feature extractor for GNNs.

On the inference front, Zhu et al. (2024) proposed a fine-tuning method for textual graphs that fuses LLMs and GNNs by means of a tunable side structure appended to each layer of the LLM, thereby improving both training efficiency and inference speed. Similarly, Duan et al. (2023) introduced SimTeG, a simple yet effective approach that augments textual graph learning by leveraging LLMs: therein, the LLM is first fine-tuned on a downstream task such as node classification, whereupon node embeddings are generated by extracting the terminal hidden states of the fine-tuned LLM. Additionally, Zhang (2023) proposed Graph-toolformer, which harnesses prompt engineering—powered by ChatGPT—to augment the graph reasoning capabilities of LLMs.

Although these approaches render LLMs more accessible for graph-related tasks, they frequently falter in fully capturing the structural intricacies intrinsic to graph data. Moreover, the performance of these methods remains heavily contingent upon effective prompt engineering—an exacting process that demands painstaking design and experimentation if optimal results are to be secured.

Graph Representation as a Learned Projector To surmount the challenges attending textual representation approaches, learned projector methods offer an alternative, transmuting graph data into embeddings or graph vectors amenable to more direct processing by LLMs. Chen et al. (2024), for instance, introduced LLAGA, a large language and graph assistant that preserves the general-purpose character of LLMs while transmuting graph structures into graph tokens—a format suited to LLM processing. In a similar vein, (Liu et al. 2024; Qin et al. 2023) infused graph structure information into a pre-trained LLM by means of tailored, disentangled graph neural network layers. Tian et al. (2024)

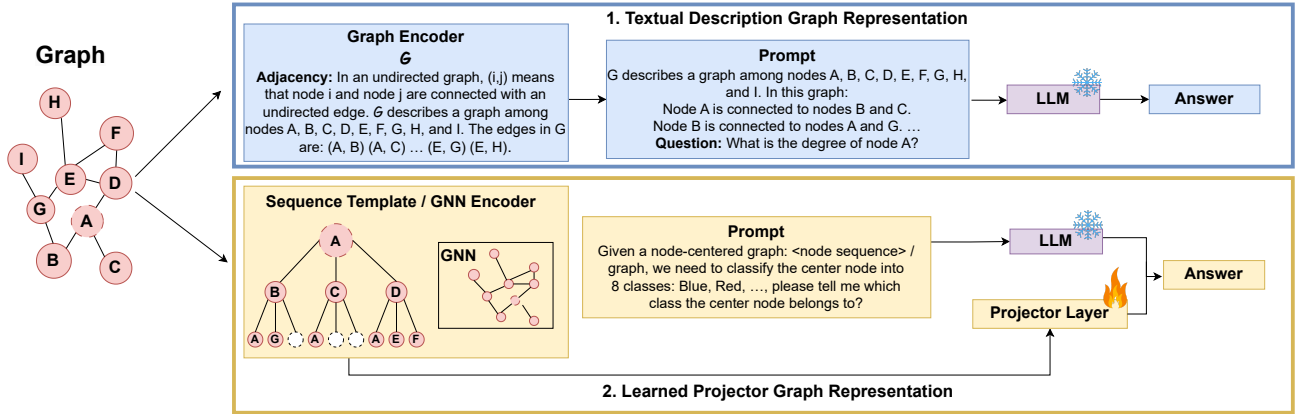


Figure 5: **Graph encoding-based adaptations of LLMs.** In the textual description graph representation, graphs are explained textually (top part). In the learned projector representation, graphs are encoded either by node templates that turns the graph into sequences or by GNNs (bottom part). Note that in both cases, the LLM is frozen. Only the projector that maps the graph into tokens is trained.

Table 7: A comparison of the graph encoding-based adaptations of LLMs.

Aspect	Textual Description Approach	Learned Projector Approach
Graph Encoding	Converts graph data (nodes, edges) into natural language descriptions.	Uses a learned projector to encode graph structures into embeddings or graph tokens.
LLM Modifications	No modification to the LLM.	No modification to the LLM.
Graph Structure Representation	Describes graph topology and relationships using textual narratives.	Encodes relational and structural information directly into vector representations.
Flexibility	Can only be used for simple graphs.	More flexible in representing complex graph structures.
Computational Cost	No additional training is required. Lower cost.	Lightweight projector is trained, typically a linear layer. Minimal cost.
Suitability for Graph Tasks	May lose some relational nuance due to limitations in textual descriptions.	Captures detailed structural information, improving performance on graph-specific tasks.

proposed the use of graph neural prompts to encode knowledge graphs, transmuting them into graph embeddings that LLMs can exploit during inference. Furthermore, Huang et al. (2023b) introduced a prompt-based node feature extractor (G-Prompt) for few-shot learning on text-attributed graphs; their method conjoins a graph adapter—a learnable GNN layer—with task-specific prompts so as to capture neighborhood information and extract informative node features.

Another promising line of work distills graph information into fixed-length prefixes by means of graph transformers, which are appended to each layer of the LLM in order to enhance its reasoning capabilities (Chai et al. 2023). Similarly, Yang et al. (2021) developed GraphFormers, which seamlessly interweaves text encoding and graph aggregation within an iterative workflow by nesting GNNs alongside the transformer blocks of a language model. Additionally, Peng et al. (2024) developed ChatGraph, an interactive interface that enables users to engage with their graphs through natural language; therein, graphs are represented as sequential paths, and the LLM is fine-tuned on the graph data so as to support graph analysis.

In this paper, we set aside projector adaptations that necessitate retraining the LLM, since they inflate computational costs considerably without occasioning commensurate performance gains. We instead concentrate on adaptation methods predicated on **frozen pre-trained LLMs** (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). This approach not only curtails computational cost but also affords greater latitude in integration across diverse LLM models; moreover, it permits us to capitalize on the extensive knowledge and capabilities of LLMs, free of the overhead of retraining, while still adapting them to specific graph tasks. Throughout this work, we employ two representative projector methods: **LLAGA** (Chen et al. 2024), which couples node-level templates with a linear projector, and **GRAPHPROMPTER** (Liu et al. 2024), which enlists a GNN-based projector to encode the graph.

This work. In this work, we set aside textual description methods, inasmuch as they frequently falter in fully capturing the structural intricacies intrinsic to graph data. Such methods are generally confined to simpler tasks—describing rudimentary graph properties such as node degree, for instance—and become infeasible for larger graphs on account of scalability constraints; in particular, the token limit imposed on prompts curtails the detailed descriptions requisite for larger graph structures, swiftly exhausting the allowable token budget. Furthermore, from a vulnerability standpoint, textual description methods introduce no unique attack surface whatsoever: because the graph is rendered as plain text descriptions (e.g., “Node A is connected to Node B”), such methods are susceptible to conventional text-based adversarial attacks rather than to the structural or feature perturbations peculiar to graph-based models. They consequently lack the complexity that would expose them to novel, graph-specific vulnerabilities, which renders them of marginal relevance to our investigation into the adversarial robustness of graph-aware LLMs.

A.2 Attacks on GNN

GNNs are beset by a diversity of attacks, among them membership inference attacks (Olatunji, Nejd, and Khosla 2021; He et al. 2021; Conti et al. 2022), which endeavor to ascertain whether a specific node figured in the model’s training set; attribute inference attacks (Gong and Liu 2018; Olatunji et al. 2023a; Duddu, Boutet, and Shejwalkar 2020), which exploit the GNN’s outputs and learned embeddings to infer sensitive or missing node attributes; property inference attacks (Zhang et al. 2022; Wang and Wang 2022), which seek to extract global properties of the graph in its entirety; adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Li et al. 2023; Dai et al. 2018; Sun et al. 2022), which strive to degrade the model’s performance by introducing subtle perturbations into the graph or the node features; and graph reconstruction attacks (Zhang et al. 2021; Olatunji et al. 2023b), which aspire to reconstruct the underlying graph structure from the learned embeddings or model outputs.

Among these, adversarial attacks are of particular moment, owing to their direct bearing upon the model’s performance. In such attacks, small yet meticulously crafted *perturbations*—the modification of node features or the alteration of edges, for instance—are introduced into the graph with the aim of appreciably degrading the GNN’s performance. These attacks fall into two types: **poisoning attacks**, which transpire during the *training phase* through manipulation of the training data, and **evasion attacks**, which transpire at *test time*, whereupon the adversary introduces perturbations into the input graph so as to mislead the model’s predictions. Zügner, Akbarnejad, and Günnemann (2018), for instance, demonstrated that adversarial perturbations of graph structures or node features can appreciably degrade the performance of GNNs; their method, **NETTACK**, marshaled a greedy optimization technique to compute the minimal perturbations that would maximize the model’s training-time loss, thereby identifying the smallest changes in the features or graph structure sufficient to disrupt the model’s predictions. Li et al. (2023) revisited the problem of adversarial attacks on graph data through the lens of data distribution, illuminating how adversarial examples can materially perturb the underlying data distribution, and proposed a novel defense mechanism to attenuate such effects. In a similar spirit, Wu et al. (2019) furnished a comprehensive analysis of adversarial examples on graph data, offering penetrating insights into both attacks and defenses; they trained their attention on the vulnerability of GNNs to small perturbations and proposed defense techniques calibrated to detect and counteract adversarial modifications, thereby fortifying model robustness. **METAATTACK** (Zügner and Günnemann 2019) mounted both poisoning and evasion attacks on GNNs by treating the graph structure as an optimizable hyperparameter. The attack modifies the graph structure directly via gradient descent, formulating the problem as a bi-level optimization task; the method exploits meta-gradients—which capture how small perturbations to the graph structure influence the attacker’s post-training loss—so as to effect precise updates that degrade the model’s performance.

In this paper, we enlist **NETTACK** (Zügner, Akbarnejad,

and Günnemann 2018) and **METAATTACK** (Zügner and Günnemann 2019) as representative adversarial attack methods, spanning both poisoning and evasion strategies, with which to conduct our investigation. Specifically, poisoning attacks assail model integrity during training, whereas evasion attacks compromise performance at inference or test time.

A.3 How NETTACK and METAATTACK Work

NETTACK effects graph structure perturbation by adding and removing edges between nodes while assiduously preserving the degree of the perturbed graph. It marshals a greedy optimization technique to compute the minimal perturbations that maximize the model’s training-time loss with respect to a surrogate model—a linearized 2-layer graph convolutional network (GCN) that approximates how alterations in the graph structure bear upon the classification outcome. The paramount objective of **NETTACK** is to inflate the model’s confidence in erroneous classifications by maximizing the distance between the logits of the perturbed graph and those of the clean graph. To this end, **NETTACK** identifies candidate edges for perturbation in the vicinity of a target node and, for each connected edge, evaluates a score function that quantifies how consequentially the modification of that particular edge would affect the target node’s classification; the edge attaining the highest score is thereupon selected, and the graph is updated by either adding or removing it. Because **NETTACK** is a targeted attack—inducing the misclassification of but a single specific node—we adapt the very method of (Zügner and Günnemann 2019) to recast it as an untargeted attack (one that compromises the model’s aggregate node classification performance), randomly selecting one test node and attacking it via **NETTACK** while accounting for all nodes in the network; we then aggregate the results over the test nodes.

METAATTACK optimizes the graph structure directly through gradient descent, treating the adjacency matrix (the graph structure) as a hyperparameter. It frames the attack as a bi-level optimization problem, wherein the inner level minimizes the training loss on the original graph while the outer level minimizes the loss on the poisoned graph. To accomplish this, **METAATTACK** enlists a surrogate model—specifically, a linearized two-layer GCN—to approximate how perturbations to the graph structure impinge upon model performance. A score function appraises the impact of each candidate perturbation, and the attack flips the sign of the meta-gradients (gradients with respect to the hyperparameters) for connected nodes so as to indicate which edges to remove. The algorithm thereafter greedily selects perturbations—adding or removing edges according to those bearing the highest scores—so as to maximize the attack’s impact.

For both methods, we impose the same unnoticeability constraint introduced by (Zügner, Akbarnejad, and Günnemann 2018), which guarantees that the graph’s degree distribution is only marginally altered; accordingly, we perturb no more than 10% of the edges.

In our poisoning attack experiments with **LLAGA**, we exploit the perturbed graph structure obtained from **NETTACK** or **METAATTACK** to generate the node-level sequences and retrain the projector that converts those sequences into

a sequence of token embeddings. Analogously, for **GRAPH-PROMPTER**, we take the perturbed graph structure, re-encode the graph by means of a graph attention network (GAT), and retrain the projector on the perturbed graph. For evasion attacks, by contrast, the graph-aware LLM is trained on unperturbed data, with perturbations introduced solely at the inference stage.

A.4 Algorithm for Node Sequence Template Injection Attacks

We present the algorithm for the node sequence template injection attack in **LLAGA** in Algorithm 1. The model erects a *fixed-shape* computational tree for each node u , sampling k neighbors per node up to depth d (lines 1–14). Whenever a node possesses fewer than k neighbors, placeholders are interposed, thereby engendering an attack surface. We exploit this surface by injecting malicious nodes into the placeholder positions through graph manipulation. We propose three strategies: *Non-Adjacent Injection (NI)* inserts nodes lying outside the 2-hop neighborhood of u (lines 21–27); *Supernode Injection (SI)* injects a single high-degree node into one placeholder (lines 29–33); and *Multiple Supernode Injection (MSI)* distributes distinct high-degree nodes across all placeholders (lines 35–43). These uninformative nodes corrupt the node sequence and thereby degrade the performance of the projection layer; indeed, their intrusion into the sequence contaminates the target’s representation, impairing the training of the projection layer.

A.5 Ablation Studies

Here, we present in full the results of the ablation studies adumbrated in the main paper.

Ablation Results: Varying Feature Perturbation In Figure 6, we chart the performance of the two graph-aware LLMs, **LLAGA** and **GRAPH-PROMPTER**, as the proportion of perturbed features is varied. We observe that escalating the fraction of perturbed features consistently erodes the accuracy of both models, a pattern that attests to their sensitivity to feature-level attacks.

Ablation Results: Does the Choice of LLM Affect Attack Performance? The results in Table 8 demonstrate that the choice of base LLM exerts only a negligible influence on both clean performance and adversarial vulnerability within the **LLAGA** framework. Across Cora, Citeseer, and PubMed, the models built upon Vicuna-7B and Llama2-7B attain all but identical accuracy under clean conditions and undergo comparable degradation under both poisoning and evasion attacks. Under **METAATTACK** evasion, for instance, Cora performance descends to 0.44 ± 0.06 for Vicuna-7B and to 0.43 ± 0.08 for Llama2-7B—a disparity that falls comfortably within the standard deviation. Analogous patterns obtain across all datasets and attack types, which indicates that attack efficacy is, to a large degree, independent of the base LLM.

A.6 Results on the ArXiv Dataset

We now evaluate our attack on the large-scale ArXiv dataset, which comprises 169,343 nodes and 1,166,243 edges and

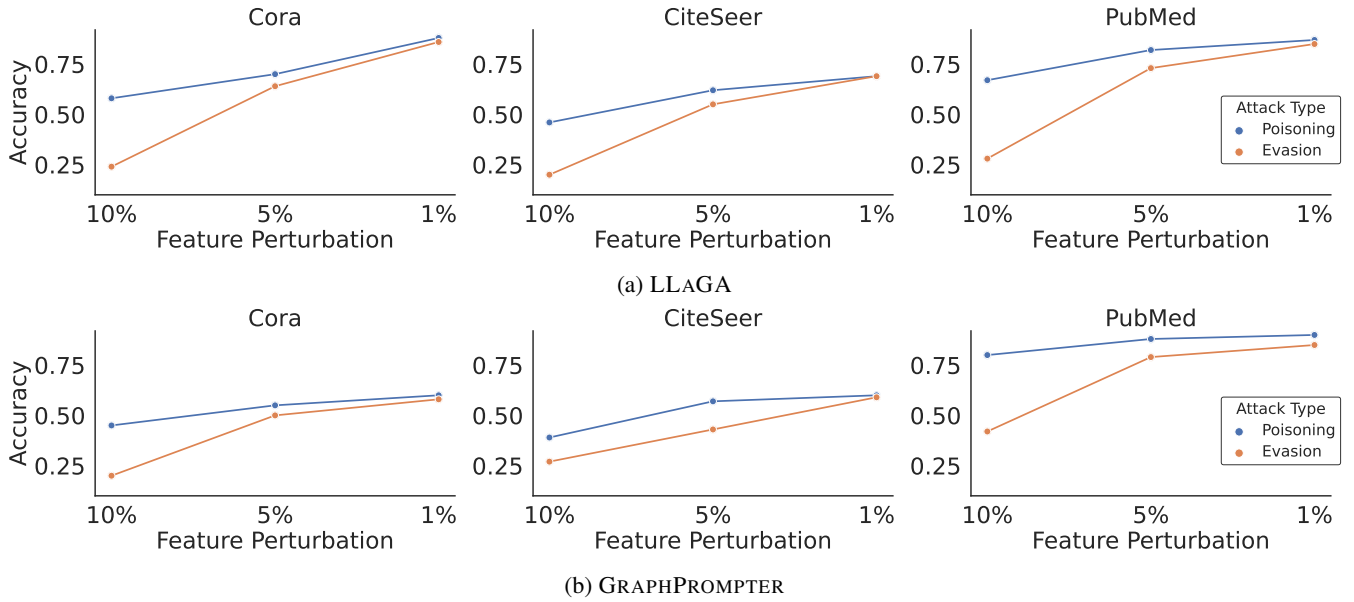


Figure 6: Performance of the graph-aware LLMs as the percentage of perturbed features is varied. A lower accuracy signifies a more successful attack.

Table 8: Impact of varying the base model on performance under clean conditions and under the various attacks, shown here for LLaMA. The results establish that the choice of base model does not appreciably affect susceptibility to attack.

Data	Base Model	Clean	Poisoning			Evasion		
			NETTACK	METAATTACK	MSI	NETTACK	METAATTACK	MSI
Cora	Vicuna-7B	0.89 ± 0.07	0.87 ± 0.04	0.79 ± 0.03	0.82 ± 0.04	0.55 ± 0.09	0.44 ± 0.06	0.30 ± 0.05
	Llama2-7B	0.90 ± 0.03	0.86 ± 0.02	0.77 ± 0.05	0.82 ± 0.02	0.56 ± 0.07	0.43 ± 0.08	0.31 ± 0.01
Citeseer	Vicuna-7B	0.71 ± 0.04	0.64 ± 0.05	0.63 ± 0.04	0.62 ± 0.02	0.59 ± 0.04	0.50 ± 0.03	0.24 ± 0.05
	Llama2-7B	0.71 ± 0.05	0.65 ± 0.03	0.60 ± 0.03	0.61 ± 0.04	0.58 ± 0.03	0.48 ± 0.05	0.26 ± 0.03
PubMed	Vicuna-7B	0.90 ± 0.04	0.89 ± 0.03	0.89 ± 0.08	0.81 ± 0.03	0.84 ± 0.05	0.73 ± 0.04	0.76 ± 0.03
	Llama2-7B	0.91 ± 0.06	0.88 ± 0.04	0.90 ± 0.02	0.80 ± 0.04	0.85 ± 0.03	0.75 ± 0.02	0.77 ± 0.02

exhibits a sparsity of 0.81. This experiment is intended to probe the efficacy of node sequence template attacks in large, real-world graphs. As reported in Table 9, we discern a pronounced gulf between the success rates of the poisoning and evasion attacks, with poisoning consistently outstripping evasion across all three injection strategies—an inversion of the pattern manifest in the remaining datasets, Cora, Citeseer, and PubMed. This intimates that ArXiv may harbor intrinsic structural idiosyncrasies that render it at once more vulnerable to poisoning and more robust to evasion. Although the sparsity of the ArXiv dataset is exceedingly low, the insertion of even a single supernode can markedly perturb model performance.

A.7 Feature Attack Examples and Defense Template

Feature Attack Examples The examples embedded in the prompt, as displayed in Appendix A.7, illustrate our adversarial attacks on authentic features drawn from the Cora dataset. The "input" denotes the text received by the LLM to which

the attack has been applied. It merits emphasis that, whereas a human reader apprehends the original and the perturbed text as visually identical or nearly so, the LLM processes a sequence of distinct Unicode characters or a permuted character order—a discrepancy that can materially alter its behavior. We render this phenomenon explicit in Table 10 for the homoglyph attack and in Table 11 for the reordering attack.

Feature Corrector Defense Template We devised a feature corrector module to attenuate the effects of imperceptible attacks—specifically, homoglyph substitutions and character reorderings. This module is realized as a prompt-based defense predicated on LLMs, with the prompt meticulously crafted to enjoin the LLM to serve as a linguistic validator dedicated to the detection and rectification of subtle, human-imperceptible perturbations.

The template for our prompt is exhibited below. By stating the task unambiguously and prescribing a clear output format, it appreciably enhances the LLM’s performance.

Prompt for Feature Corrector

You are a linguistic validator. Your task is to detect and correct any subtle, imperceptible text perturbations such as homoglyph substitutions (e.g., replacing "o" with "0") and character reorderings. These changes are often used to attack machine learning models. Given a text string, carefully analyze it. If you identify any perturbations, provide the corrected, original-looking version of the text. If no perturbations are found, return the original text unchanged.

Example 1 (Homoglyph Substitution):

Input: Title: The mgprir heuristic fr discvring prtin squnc pttrens Abstract: Svrl cmputr lgrithms fr discvring pttrens in grups f prtin squncs r in us tht r bsd on fitting th prmtrs f sttisticl mdl to grup of rlted squncs.

Output: Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Example 2 (Character Reordering):

Input: Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dscribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiottn of the apclpiation of three machine larening algorithms-lR, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.

Output: Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-lR, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

Example 3 (Mixed Homoglyph & Reordering):

Input: Title: Submitted to NIPS96, Sctin: Applications. Prfrnc: Oral prsentatin Reinforcement Lrning for Dynmic Channl Allocation in Abstract: In ellulr tlphne systms, n imrtnt prblm is to dynmiclly alloct th cmmunictin rsurc (chnnls) so as to mximize srvc in stchstic cllr nvirnmt. This prblem is naturally frmultd s dynmic prgrmming prblm and we us a reinforcement lerning (RL) mthd to find dynmic chnnl llection plicies that ar bttr than prvius heuristic slutins.

Output: Title: Submitted to NIPS96, Section: Applications. Preference: Oral presentation Reinforcement Learning for Dynamic Channel Allocation in Abstract: In cellular telephone systems, an important problem is to dynamically allocate the communication resource (channels) so as to maximize service in a stochastic caller environment. This problem is naturally formulated as a dynamic programming problem and we use a reinforcement learning (RL) method to find dynamic channel allocation policies that are better than previous heuristic solutions.

Your Task: Text to correct: {perturbed_text_here}

Corrected Text:

Figure 7: The prompt template employed by the LLM-based feature corrector module. It delineates unambiguously the role, the task, and the expected output format prescribed for the LLM.

Algorithm 1: Injection Attacks on Neighborhood Detail Template for Obtaining Node Sequence.

Input: Graph $G = (V, E)$, target node u , tree depth d , children size k
Output: Modified graph G' with injected nodes

```

1: procedure NEIGHBORHOODDETAILTEMPLATE( $u, d, k$ )
2:    $T \leftarrow \emptyset$  ▷ Initialize empty tree
3:    $T.\text{root} \leftarrow u$ 
4:   for level  $\ell = 1$  to  $d$  do
5:     for each node  $v$  at level  $\ell - 1$  do
6:        $N_v \leftarrow \text{Sample}(\mathcal{N}(v), k)$  ▷ Sample  $k$  neighbors
7:       if  $|N_v| < k$  then
8:         Fill remaining slots with placeholders
9:       end if
10:      Add  $N_v$  as children of  $v$  in  $T$ 
11:    end for
12:  end for
13:  return  $T$ 
14: end procedure

15: procedure ATTACK( $G, u, d, k, \text{AttackStrategy}$ )
16:    $G' \leftarrow G$  ▷ Create copy of original graph
17:    $T \leftarrow \text{NEIGHBORHOODDETAILTEMPLATE}(u, d, k)$ 
18:    $P \leftarrow \text{GetPlaceholders}(T)$ 
19:   if  $|P| > 0$  then
20:      $\mathcal{N}_2 \leftarrow 2\text{-hop neighborhood of } u$ 
21:     if  $\text{AttackStrategy} = \text{NI}$  then
22:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
23:        $V_{\text{inject}} \leftarrow \text{Sample}(V_{\text{non-adjacent}}, |P|)$ 
24:       for each placeholder position  $p \in P$  do
25:          $v \leftarrow \text{Next node from } V_{\text{inject}}$ 
26:         Add edge  $(\text{parent}(p), v)$  to  $G'$ 
27:       end for
28:     else if  $\text{AttackStrategy} = \text{SI}$  then
29:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
30:        $s \leftarrow \text{GetHighestDegreeNode}(V_{\text{non-adjacent}})$ 
31:       for first placeholder position  $p \in P$  do
32:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
33:       break ▷ Only use first placeholder
34:     end for
35:     else if  $\text{AttackStrategy} = \text{MSI}$  then
36:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
37:        $S \leftarrow \text{GetNodesOrderedByDegree}(V_{\text{non-adjacent}})$ 
38:       for  $i = 1$  to  $|P|$  do
39:          $s \leftarrow \text{Get } i\text{-th node from } S$ 
40:          $p \leftarrow \text{Get } i\text{-th placeholder from } P$ 
41:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
42:       end for
43:     end if
44:   end if
45:   return  $G'$ 
46: end procedure

```

A.8 Detailed Discussion

The findings of our work occasion several critical insights into the vulnerabilities and design considerations attending

Table 9: Results of the node sequence template injection attack across the various datasets and methods, now inclusive of the large-scale ArXiv dataset. *NI*, *SI*, and *MSI* denote, respectively, the non-adjacent, single-supernode, and multiple-supernode injection attacks. Clean designates the performance of the model in the absence of any attack.

Data	Attack	NI	SI	MSI
Cora	Clean	—	0.89 $\pm$ 0.07	—
	Poisoning	0.86 $\pm$ 0.03	0.84 $\pm$ 0.06	0.82 $\pm$ 0.04
	Evasion	0.42 $\pm$ 0.08	0.36 $\pm$ 0.05	0.30 $\pm$ 0.05
Citeseer	Clean	—	0.71 $\pm$ 0.04	—
	Poisoning	0.69 $\pm$ 0.05	0.66 $\pm$ 0.03	0.62 $\pm$ 0.02
	Evasion	0.36 $\pm$ 0.06	0.30 $\pm$ 0.04	0.24 $\pm$ 0.05
PubMed	Clean	—	0.90 $\pm$ 0.04	—
	Poisoning	0.88 $\pm$ 0.04	0.85 $\pm$ 0.05	0.81 $\pm$ 0.03
	Evasion	0.85 $\pm$ 0.07	0.82 $\pm$ 0.04	0.76 $\pm$ 0.03
ArXiv	Clean	—	0.73 $\pm$ 0.02	—
	Poisoning	0.50 $\pm$ 0.03	0.43 $\pm$ 0.04	0.42 $\pm$ 0.03
	Evasion	0.71 $\pm$ 0.06	0.69 $\pm$ 0.02	0.60 $\pm$ 0.03

graph-aware LLMs. As the assimilation of graph-structured data into LLMs continues to gather momentum for graph tasks, it becomes imperative to interrogate not merely the performance dividends of these novel architectures but their vulnerabilities as well. Our systematic investigation of adversarial attacks on two representative graph-aware LLMs—LLAGA and GRAPH PROMPTER—lays bare significant weaknesses, most conspicuously when the models are subjected to the poisoning and evasion attacks traditionally wielded to undermine GNNs. Our principal observations are expounded below.

Impact of Graph Encoding on Adversarial Vulnerability The cardinal observation issuing from our results is that the manner in which graph structure is integrated into LLMs can inadvertently heighten their susceptibility to adversarial attack. Specifically, the node sequence template employed in LLAGA appears to beget a vulnerability that adversaries may exploit to precipitate substantial degradation in model performance. This attack surface, peculiar to LLAGA, throws into relief a critical tradeoff in graph-aware LLM design: while node-level templates furnish an effective vehicle for incorporating graph structure and confer superior performance on graph tasks, they concomitantly expose the model to new attack vectors absent from traditional GNNs. GRAPH PROMPTER, by contrast, which relies on a GNN-based projector for graph encoding, evinces comparatively greater resilience to these attacks—thereby underscoring the prospective merits of hybrid architectures that marshal the complementary strengths of both LLMs and GNNs.

Sensitivity of Graph-aware LLMs to Feature Perturbations Beyond the vulnerabilities inherent in the graph encoding techniques, our results further illuminate the potency of feature perturbation attacks across both graph-aware LLMs. Notwithstanding their divergent strategies for encoding graph structure, both LLAGA and GRAPH PROMPTER be-

Table 10: An illustration of the imperceptible homoglyph attack. Although the perturbed text is processed by the LLM as a sequence of distinct characters, its rendered form remains nearly indistinguishable to a human observer, as evidenced in the image.

Original Feature	What LLM Sees	What a Human Sees
Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.	Title: The mgpprir huristic fr discvring prtln squnc ptttrns Abstract: Svrl cmputr lgrithms fr discvring pttrns in grups f prtln squncs r in us tht r bsd on fitting th prmtrs f sttistical mdl to grup of rltd squncs.	Title: The megaprior heuristic for discovering protein sequence patterns. Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Table 11: An illustration of the imperceptible reordering attack. Although the perturbed text is processed by the LLM as a sequence of distinct characters, its rendered form remains nearly indistinguishable to a human observer, as evidenced in the image.

Original Feature	What LLM Sees	What a Human Sees
Title: Applications of machine learning: a medical follow up study. Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.	Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dsceribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiotn of the aplcpiation of three machine larening algorithms-1R, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.	Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

tray a pronounced sensitivity to imperceptible perturbations of the node features. This finding brings into focus a broader tendency of the LLM paradigm: although such models excel at handling textual data, their robustness to adversarial perturbations—especially those directed at node features—remains a critical open challenge. The capacity of adversaries to degrade model performance by subtly altering node attributes while eluding detection casts doubt upon the integrity of graph-aware LLMs, above all in security-sensitive applications. While text sanitization techniques, such as those canvassed by (Boucher et al. 2022), offer potential recourse, contending with imperceptible attacks—homoglyph-based manipulations among them—remains an intricate undertaking. As we propose, enlisting LLMs as a corrector constitutes

one avenue for mitigating such vulnerabilities, prompting the model to detect and repair suspicious or malformed inputs prior to downstream processing.

Data-dependent Variability of Poisoning and Evasion Attacks on Graph-aware LLMs Our findings reveal that the efficacy of poisoning and evasion attacks against graph-aware LLMs is by no means uniform across datasets. Whereas the attacks prove particularly effective on the Cora, Citeseer, and PubMed datasets, we discern a considerably more nuanced interplay between the attack strategies and the graph structure on the ArXiv dataset. This variability accentuates the salience of dataset characteristics in appraising the vulnerabilities of graph-aware LLMs. As these models are increasingly deployed across heterogeneous real-world settings, their vulner-

abilities must be adjudged not solely in terms of algorithmic design but also with reference to the particular domains and data to which they are applied.

Implications for Graph-aware LLM Development The vulnerabilities laid bare in this study carry broader ramifications for the future development and deployment of graph-aware LLMs. Given their escalating adoption for tasks once the exclusive province of GNNs, it is incumbent upon designers to weigh the tradeoffs between performance and vulnerability in the conception of these paradigms. As graph-aware LLMs continue to evolve, there emerges a pressing need for more robust encoding techniques and for defenses against adversarial manipulation, most acutely in the setting of feature perturbations. Moreover, elucidating how adversarial attacks transfer from GNNs to graph-aware LLMs opens fresh avenues of research into safeguarding these models against sophisticated adversaries.

Difficulty in Applying Existing GNN defenses Against Adversarial Attacks on Graph-aware LLMs **Defenses on adversarial attacks based on feature perturbation.** Adversarial attacks on GNNs have concentrated overwhelmingly on structural perturbations, since feature-based perturbations typically fail to sway the predicted class of the target node (Dai et al. 2018; Wu et al. 2019; Zügner and Günnemann 2019) and hence constitute ineffective attack strategies. In consequence, and owing to their limited effectiveness, there exist *no known defenses against feature-based attacks on GNNs*. As demonstrated in the main paper, however, attacks predicated on feature perturbation prove decidedly effective within the graph-aware LLM paradigm. This engenders a novel challenge peculiar to the graph-aware LLM framework, wherein feature-based attacks—hitherto inert against GNNs—can now successfully manipulate model behavior. Such a shift accentuates the need to develop new defenses that contend simultaneously with structural and feature-based adversarial perturbations. While text sanitization techniques—for instance, the deployment of optical character recognition (OCR) as a preprocessing step for encoding text bound for the LLM—offer potential recourse, redressing these imperceptible attacks, homoglyph-based manipulations among them, remains an intricate challenge, inasmuch as OCR is imperfect and misconstrues homoglyphs at a rate exceeding that for unperturbed text (Boucher et al. 2022). It is this predicament that motivated our design of a self-correcting mechanism for feature perturbation grounded in LLMs.

Defenses on adversarial attacks based on structural perturbation. Owing to the transferability of structure-based adversarial attacks from GNNs to graph-aware LLMs—particularly in the case of evasion attacks—one might naturally anticipate that defenses devised for GNNs would prove equally efficacious in this paradigm. This expectation, however, does not withstand scrutiny. Unlike traditional GNN settings, *graph-aware LLMs are additionally susceptible to feature perturbations, a susceptibility that renders these defenses ineffective*.

One such defense mechanism in GNNs renders the adjacency matrix trainable, permitting the edge weights to adapt during training so as to attenuate adversarial perturbations. This approach apports lower weights to edges joining dissimilar nodes and higher weights to those linking nodes of greater feature similarity. Several **preprocessing-based** defenses have been erected upon this foundation—among them the excision of edges between nodes of low feature similarity (Wu et al. 2019) and the introduction of randomized edge dropping (Dai et al. 2018). Such defenses, however, depend heavily on clean, unperturbed node features to compute reliable similarity scores; once the node features are themselves perturbed, as under our attacks, these defenses become ineffective and may even aggravate the problem by forging incorrect edges. We demonstrate this effect on a randomly chosen node in the Cora dataset (Figure 8). As can be observed, whereas the clean feature similarity graph preserves a portion of the original edges even as it introduces a number of spurious connections, the edges reconstructed from the perturbed features preserve none of the original connections whatsoever, yielding reconstructions that are entirely erroneous.

A further line of work concentrates on decoupling the model’s reliance on the graph structure. (Shanthamallu, Thiagarajan, and Spanias 2021), for instance, proposed a defense that trains a surrogate predictor contingent solely upon node features and enlists an uncertainty-matching strategy to extract graph information from the GNN. In a kindred vein, Wu et al. (2021) introduced a co-training framework that trains submodels independently on the feature and the structural views, enabling knowledge distillation through the incorporation of confidently classified unlabeled data into the training set. Yet these approaches, too, presuppose access to unperturbed node features—a presupposition that collapses in our setting, where both structure and features are adversarially perturbed. Consequently, structural defense strategies developed for GNNs are not directly transposable to the graph-aware LLM paradigm, which instead demands novel defenses tailored expressly to it.

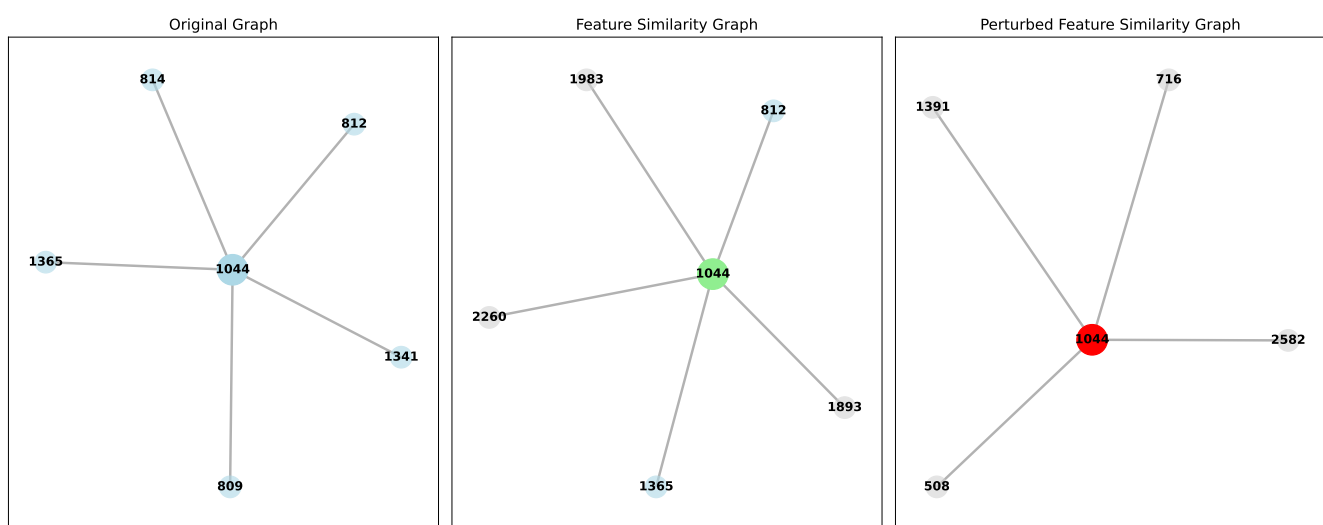


Figure 8: Edges reconstructed for a randomly selected node, 1044. **Original (left)** depicts the ground-truth edges from the original graph. **Feature Similarity (middle)** shows the edges reconstructed from clean node features, wherein blue nodes signify connections preserved from the original graph. **Perturbed Feature Similarity (right)** shows the edges reconstructed from perturbed node features.